

8-1 · Airflow 서비스 구축 · 이론 1H

오케스트레이션과 Airflow 개요

cron 이 모르는 것 — 순서·실패·기록

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 오케스트레이션이 cron·수동 실행의 어떤 한계를 푸는지 말할 수 있다
- ② 웹서버·스케줄러·워커·메타데이터 DB 의 역할을 비유로 구분한다
- ③ DAG 가 무엇의 약자이고 왜 파이썬 코드로 쓰는지 안다
- ④ Airflow 가 왜 컨테이너 여섯 개로 뜨는지 설명한다

다음 1.5H 실습에서 이 여섯 컨테이너를 공식 `compose` 로 직접 띄웁니다

1. cron 으로는 왜 부족한가

cron 은 시각만 안다 — 단계가 이어지면 부족

도구 선택의 기준

단계가
이어지는가

- 1 단계가 다섯 개만 되어도**
「2번이 끝나야 3번을 돌리는데, 2번이 실패하면?」
- 2 cron 은 시각만 안다**
앞 단계가 끝났는지, 실패했는지 모른다
- 3 cron 이 나쁜 것은 아니다**
작업 하나를 정해진 시각에 돌리는 일이라면 cron 이 옳다

1. cron 으로는 왜 부족한가

필요한 것 일곱 — cron 과 Airflow 비교

필요한 것	cron	Airflow
정해진 시각에 실행	된다	된다
앞 작업이 끝나야 시작	안 된다 — 시간으로 추측	의존성으로 보장
실패한 단계만 재시도	전부 다시	그 태스크만
실행 이력·로그 보관	직접 만들어야	기본 제공
실패 알림	직접 만들어야	설정으로
과거 날짜 소급 실행	안 된다	backfill
누가 언제 무엇을 고쳤나	알 수 없다	DAG 가 코드 → git

작업 사이에 순서가 생기는 순간부터 오케스트레이션이 필요해진다

구성 요소 넷 — 비유로 기억

01

웹서버 = 상황판

DAG 목록·이력·로그를
사람이 보는 곳

02

스케줄러 = 알람시계

시각이 되면 DAG 를
깨워 큐에 넣는다

03

워커 = 손

큐에서 태스크를
꺼내 실행한다

04

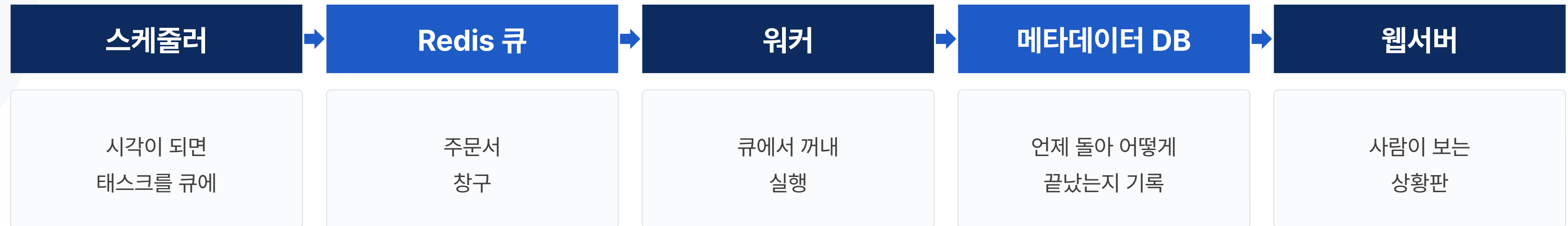
메타데이터 DB = 일지

실행 이력의 정보
깨지면 통째로 사라진다

메시지 큐(Redis)는 스케줄러와 워커 사이의 주문서 창구 · 트리거러는 기다리는 일을 워커를 붙잡지 않고 처리

2. Airflow 의 구성 요소

스케줄러가 넣고, 워커가 꺼내고, DB 에 적힌다



스케줄러는 Airflow 의 심장 · 워커는 여럿을 둘 수 있다 · 큐는 실행 방식(Executor)에 따라 없을 수도

메타데이터 DB 가 정본 — `docker compose down --volumes` 가 이것을 지우므로 함부로 쓰지 않는다

3. DAG

DAG — 오늘은 이름만

1 Directed Acyclic Graph
방향이 있고 순환이 없는 그래프

2 요리 레시피 순서도
재료 손질 → 볶기 → 담기 — 담은 음식을 다시 손질하러 돌아가는 화살표는 없다

3 파이썬 코드로 정의
그래서 git 으로 버전 관리하고 리뷰할 수 있다 — 8-2 에서 자세히

Airflow 의 모든 워크플로는

**순서도
하나로**

4. 왜 컨테이너 여러 개인가

한 묶음으로 떠야 Airflow — 여섯 컨테이너

서비스	하는 일
airflow-webserver	사람이 보는 화면 (8080)
airflow-scheduler	시각을 보고 태스크를 큐에 넣는다
airflow-worker	큐에서 꺼내 실제로 실행한다
airflow-triggerer	기다리는 일을 전담
postgres	메타데이터 DB — 실행 이력의 정본
redis	스케줄러와 워커 사이의 큐

1주차에서 배운 Compose 가 여기서 값을 한다

구성 요소가 각각 프로세스로 도는 작은 분산 시스템 — 그래서 공식 docker-compose 파일 하나로 묶어 기동하는 것이 표준 입문 방법

이 1시간에서 꼭 가져갈 것

- 1 작업 사이에 순서가 생기면 cron 으로는 부족하다
- 2 Airflow 는 순서·재시도·기록을 대신 관리한다
- 3 웹서버는 상황판, 스케줄러는 알람시계, 워커는 손, 메타데이터 DB 는 일지
- 4 DAG 는 방향 있고 순환 없는 순서도이며 파이썬 코드로 쓴다
- 5 Airflow 는 여러 컨테이너가 함께 떠야 도는 작은 분산 시스템이다

8-1 · 이론 1H 마무리

다음은 Airflow 기동 실습 (1.5H)

공식 docker-compose 구성을 받아 먼저 읽고, 초기화 → 기동 → 상태·로그 확인까지 직접 해 봅니다

✓ Docker 메모리 확인 · 안 쓰는 컨테이너 stop

✓ compose 파일을 grep 으로 먼저 읽기

✓ airflow-init 뒤 up -d 로 기동

✓ stop → start 로 데이터 유지 확인

준비할 것

Docker 메모리 4GB(권장 8GB) 확인